



《软件测试》

第六章 软件测试的度量

授课人：高利鹏

日期：2025年12月16日



目 录

CONTENTS

01

软件测试度量简介

02

度量方法及其应用

03

常见的度量类型

04

小结

学习目标

- ◆ 了解软件测试度量的概念
- ◆ 掌握软件测试度量的方法和应用
- ◆ 了解常用的软件测试度量类型

目 录

CONTENTS

01

软件测试度量简介

02

度量方法及其应用

03

常见的度量类型

04

小结

目 录

CONTENTS

1.1 度量的目的

1.2 测试度量的难度

1.3 测试人员工作质量的衡量

软件度量

- **度量**是指对一个系统或过程的**某些属性**方面的衡量。
- **软件的度量**包括对软件产品自身的测量，以及产生软件产品过程的测量。
- 为了评估软件过程、产品，以及服务而使用的度量称作**软件度量**。

软件测试度量

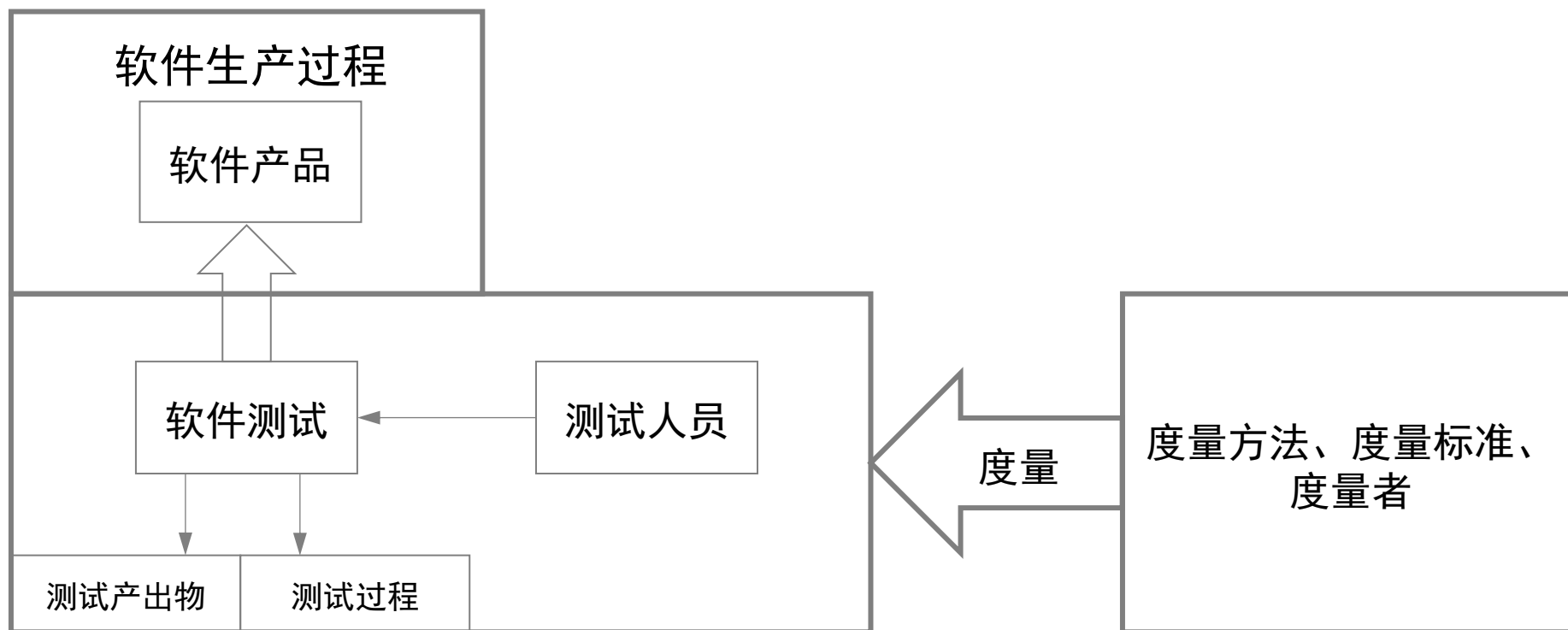
- 软件测试的度量则包括对软件测试的产出物的测量，以及测试的过程的测量。
- 软件测试中的度量一般有如下目的：
 - 判断软件测试的有效性。
 - 判断软件测试的完整性。
 - 判断所测试的软件产品的质量。
 - 分析和改进软件测试过程。

度量框架

- 度量的数据构成一个层次化的体系，就是度量框架。
- 框架的上层是度量指标（Factor），下层是直接度量（Metrics）。
 - 度量指标
 - 产品或过程的特征
 - 根据直接度量计算
 - 直接度量
 - 直接收集到的数据

度量的目的

- 软件度量与软件测试度量，以及测试人员的关系图如下图所示：

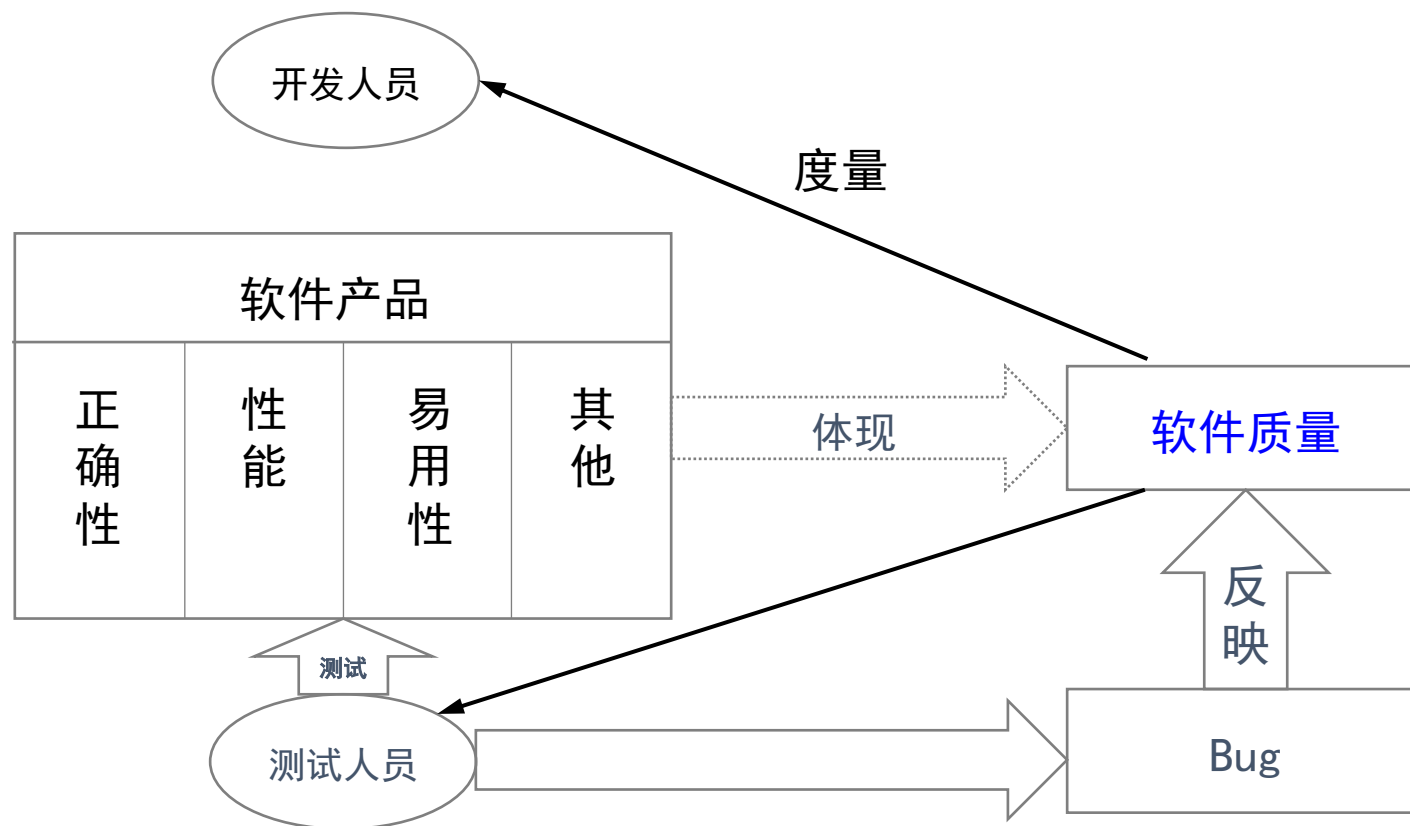


度量的目的

- 测试的度量应该遵循以下原则：
 - 要制定明确的**度量目标**；
 - 度量标准的定义应该具有**一致性、客观性**；
 - 度量方法应该尽可能**简单、可计算**；
 - 度量数据的收集应该尽可能**自动化**。

度量的目的

- 开发人员、测试人员及软件产品之间的关系如下图所示：



目 录

CONTENTS

1.1 度量的目的

1.2 测试度量的难度

1.3 测试人员工作质量的衡量

测试度量的难度

- 软件测试是用于度量软件质量的一种手段，通过测试发现产品缺陷，进而评估软件的整体质量。
- 那么测试本身的质量如何度量呢？
- 开发人员开发产品、测试人员测试产品，真实可见的是软件产品的质量，软件的质量可直接反映开发人员的工作效率。但是否能反映测试人员的工作效果呢？

测试度量的难度

例子：

- 假设A项目的研发时间充裕，同时也配备了充足的资源，如优秀的设计架构师和开发人员，并且能够做到与客户充分地沟通交流；
- B项目则有点窘迫，进度明显较慢，开发人员虽然很负责任，但是士气明显有些低落。
- 如果A项目在做测试，不出意外的话最终的产品结果是质量过关，测试人员受到好评。但是，如果在B项目做测试，即使测试人员很努力，也不可能做的很好。

测试度量的难度

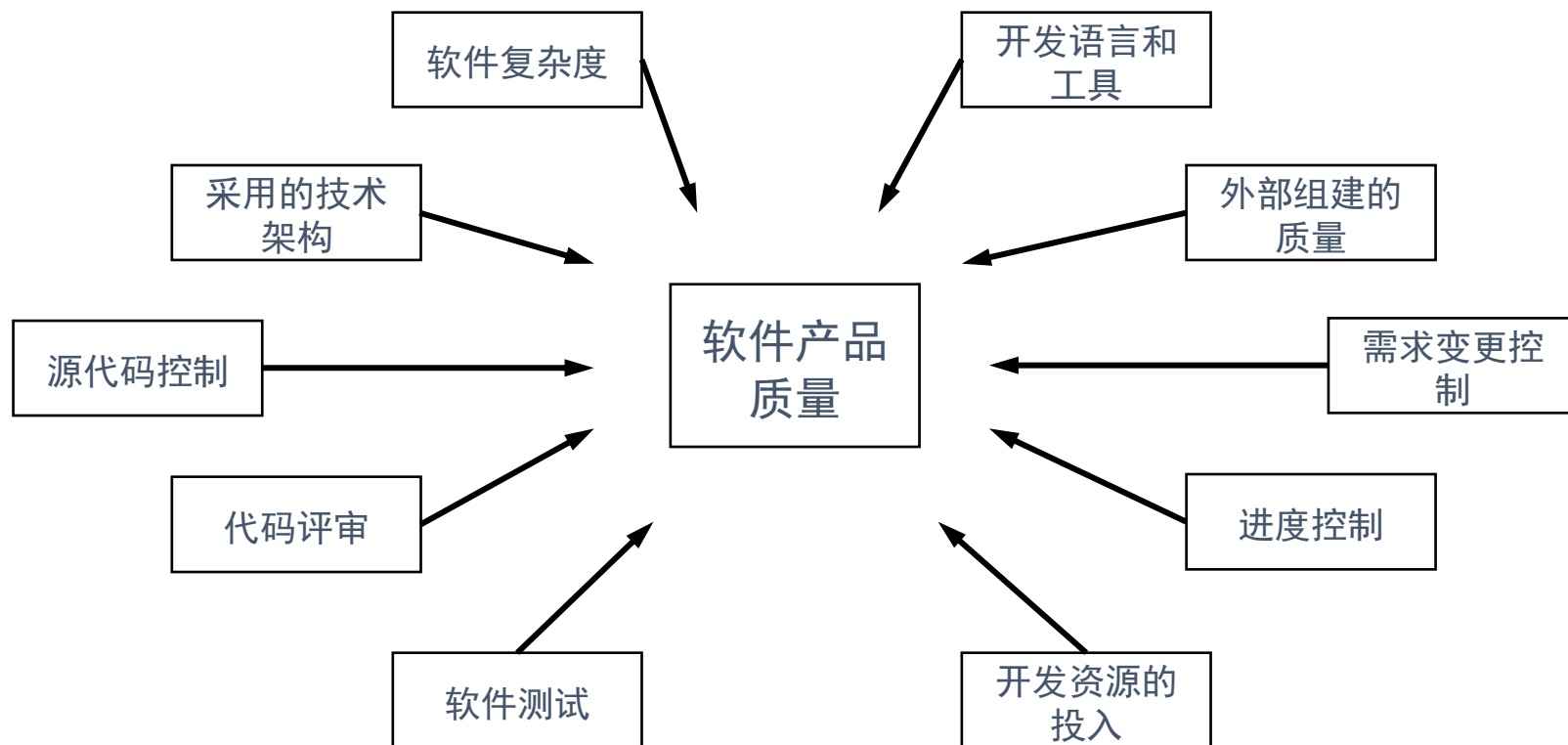
从这个例子，可以得到两点启示：

(1) 测试不能提高质量，软件的质量是固有特性，测试人员只能通过测试来评估产品质量，产品质量的提高有赖于开发人员的努力。

(2) 测试人员的工作成果不能从软件的产品质量或软件的最终成果得到科学的评估。不能用最终产品来决定一个测试是否成功，或者测试人员是否优秀，要考虑测试过程的更多因素。

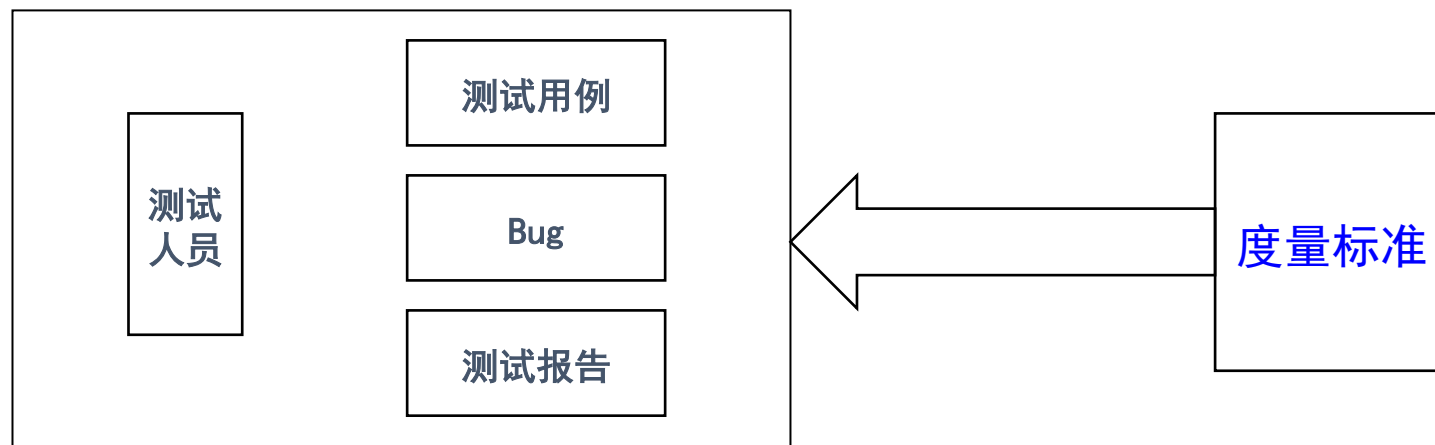
测试度量的难度

- 影响产品质量的因素如下图所示。



测试度量的难度

- 软件测试只是影响产品质量的其中一个因素，软件测试做的不好，会影响质量的改进与提高，但是绝对不能依赖软件测试来提高产品质量，而是要更多地从其他方面投入，例如，充分估计软件的复杂度，投入足够的开发资源，选用合理的体系结构和开发工具、适当的代码评审等。
- 测试度量的难度在于，不能直接从产品的质量反映测试的效果。对于测试的度量，应该从软件产品的度量转移到测试产出物的度量，以及测试过程的度量。



目 录

CONTENTS

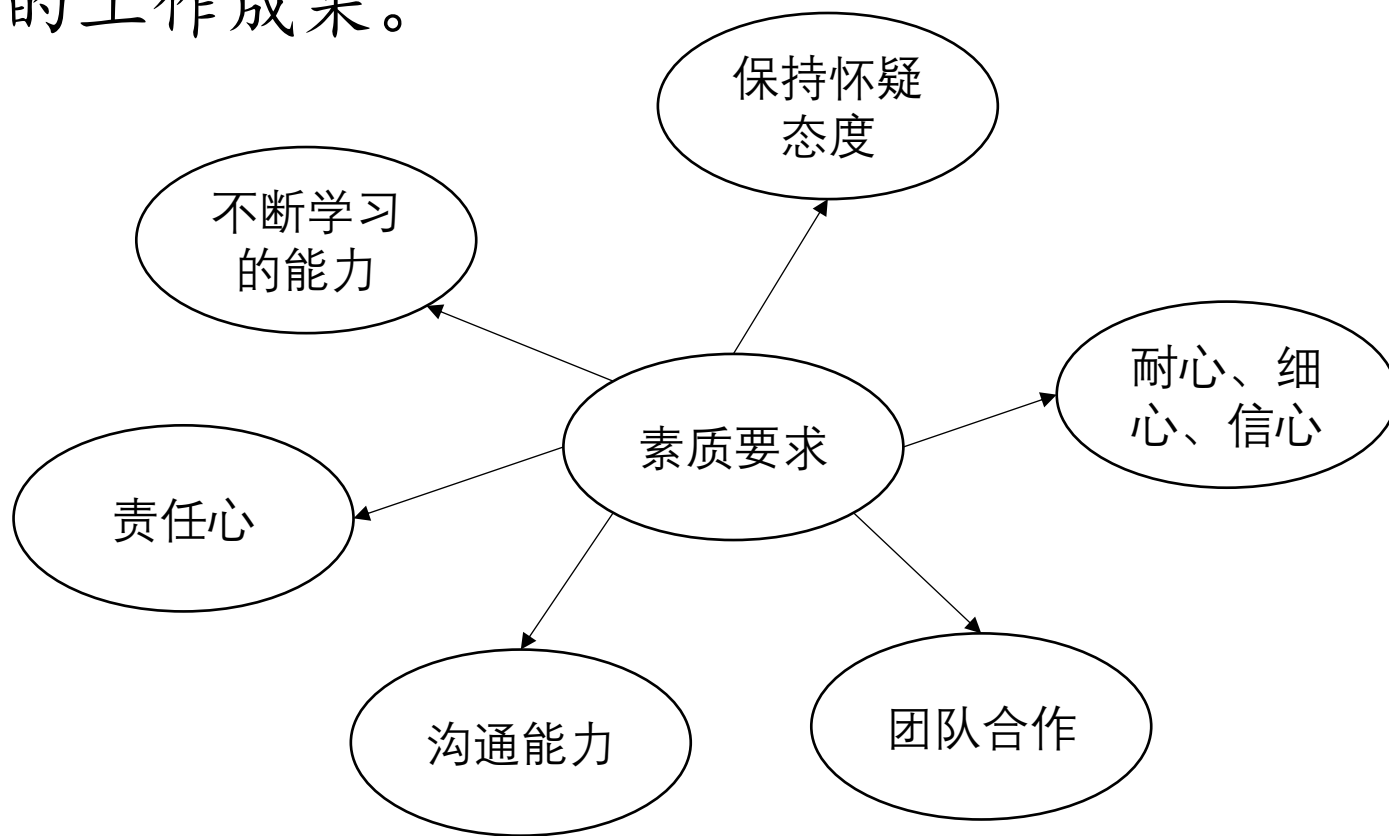
1.1 度量的目的

1.2 测试度量的难度

1.3 测试人员工作质量的衡量

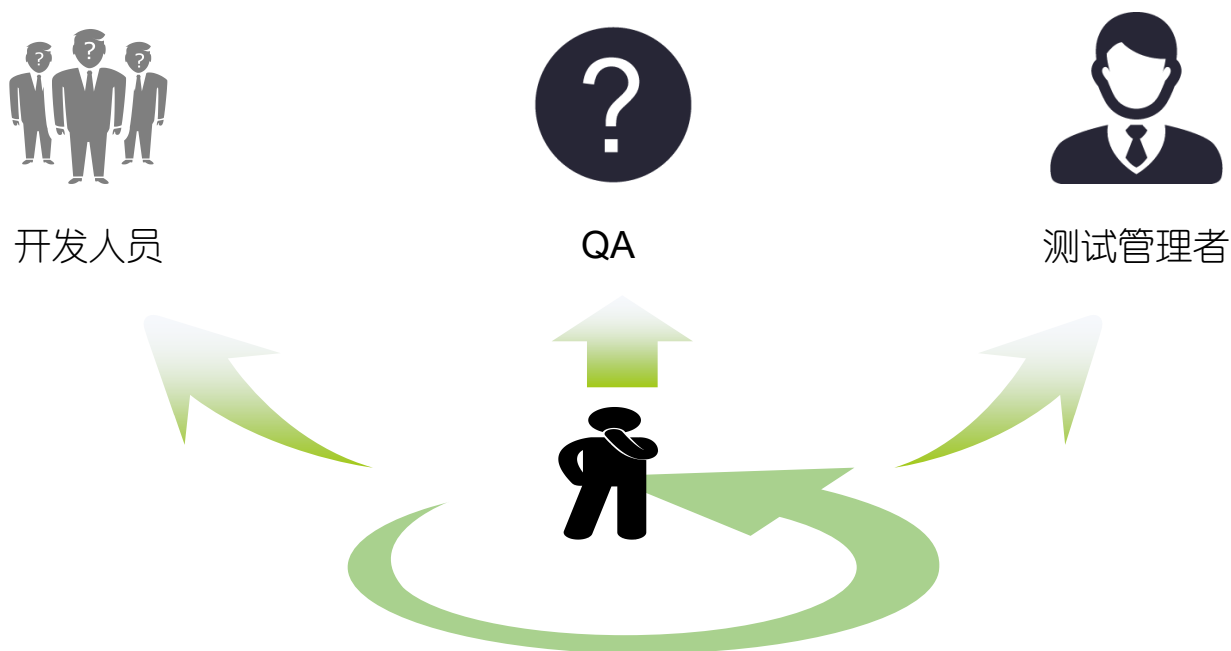
测试人员工作质量的衡量

- 测试人员是测试过程的核心人物，测试人员的工作质量会极大地影响测试的质量以及产品的质量。测试人员可以对别人的工作作出侧面的评价，因为可以通过测试人员的测试结果来衡量开发人员的工作成果。



测试人员工作质量的衡量

- 那么，谁来对测试人员的工作作出评价呢？



测试人员工作质量的衡量

● 测试人员的技能要求：

1. 业务知识：

测试人员对业务知识了解得越多，测试就越贴近用户的实际需求。

并且测试发现的软件缺陷也是用户非常关注的软件缺陷，同时还是项目经理、开发人员都会认为很重要的软件缺陷。

2. 产品设计知识：

测试人员对软件产品相关的信息了解得越多，对测试越有利；对软件产品设计、软件架构方面的信息了解得越多，越有利于把测试进行得更加深入，测试的范围也会越广。

3. 软件架构知识：

对于产品知识了解得越多，测试就能越深入产品的核心位置。

例：如果一个系刚成立，尚无学生，我们就无法把这个系及其系主任的信息存入数据库。

4. 统一建模语言（Unified Modeling Language, UML）：

现在，大部分软件开发组织都在使用UML指导设计和开发。

测试人员工作质量的衡量

5. 测试工具：常用功能自动化测试工具

厂商	工具名称
HP Mercury	QuickTest Pro
Micro Focus	TestPartner
Micro Focus	SilkTest
IBM Rational	Robot
IBM Rational	Functional Tester
Parasoft	WebKing
Oracle	e-Tester
AutomateQA	TestComplete
SeaStone Software	EggPlant
Microsoft	Visual Studio Test Edition
Software Research	eValid
开源	Selenium
开源	WebInject
开源	Watir

测试人员工作质量的衡量

6. 不同的测试手段和测试工具：

不同的项目采用的技术手段一般不一样，采用的平台、语言、开发工具、控件一般也不尽相同。

例如，同样是性能测试，在项目A中能够使用LoadRunner录制脚本，但是到了项目B就录制不下来。

7. 开发工具：

测试人员有必要掌握开发工具的一些基本操作，这样对测试过程和问题重现等会起到事半功倍的作用。而且，如果进行白盒测试，对开发工具的掌握就更不可或缺了

8. 用户心理学：

测试应该始终站在用户、使用者的角度去考虑问题，而不应该站在开发人员、实现者的角度考虑问题。

9. 界面设计中的3种模型：

设计者模型、实现者模型和用户模型。

测试人员工作质量的衡量

10. 人机交互认知心理学：

人机交互是一个从用户体验的角度出发考虑用户感受的过程。考虑到用户心理学和认知科学等，测试人员不得不根据以下基本原则指导界面测试。

11. 编程技能：

编程技能未必是必不可缺的技能，但是如果掌握基本的编程技巧，则会对测试有事半功倍的效果。

12. 脚本语言：

不需要追求精致的语言应用，不追求完美的可重用性，甚至在有些时候也不会追求性能和效率，但是归根结底，需要的是快速、能解决实际的问题

13. 文档能力：

一个优秀的测试人员应该善于利用这些书面的沟通方式来表达自己的观点、体现自己的能力和价值

目 录

CONTENTS

01

软件测试度量简介

02

度量方法及其应用

03

常见的度量类型

04

小结

目 录

CONTENTS

2.1 度量Bug的数量

2.2 加权法度量缺陷

2.3 Bug的定性评估

2.4 Bug综合评价模型

2.5 测试覆盖率统计

度量Bug的数量

- 软件测试的度量方法有很多，根据测试产出物的不同需要定义不同的度量方法。对测试人员的度量则要考虑更多因素。
- Bug的数量能够说明什么问题？对于这个问题的思考确实存在有一些矛盾心理。

利用Bug数量来考核测试效率。

如果在考核的过程中发现的漏洞越多，那么说明这个测试人员的测试效率越高，测试能力越强。



发现Bug数量的多少并不能完全证明测试人员的能力。但是如果把Bug数量加上一些前置条件（如Bug的严重程度），就会有一定的说明意义。

度量Bug的数量

例子：在同一个项目中，A和B两个测试人员参与同样的测试工作，统计出如下数据：

测试人员A：发现级别为1的缺陷100个，级别为2的缺陷150个，级别为3的缺陷250个；

测试人员B：发现级别为1的缺陷10个，级别为2的缺陷200个，级别为3的缺陷350个；

- 虽然测试人员B发现的Bug比测试人员A要多一些，但是不会认为测试人员A比测试人员B逊色，甚至认为测试人员A要表现的更加优秀一些，因为A发现了大部分的严重的Bug，修改这些严重的Bug对于用户来说是至关重要的，因此测试人员A发挥的价值要相对大一些。

目 录

CONTENTS

2.1 度量Bug的数量

2.2 加权法度量缺陷

2.3 Bug的定性评估

2.4 Bug综合评价模型

2.5 测试覆盖率统计

加权法度量缺陷

- 仅凭Bug数量的多少并不能完全说明测试人员的能力，正确的做法应该是在Bug数量度量的基础上加入以下前提条件：
 - 给缺陷加权；
 - 度量筛选后的Bug。
- 按缺陷的严重程度分级，然后每一个级别的权值由高到低对应，如图所示。

缺陷严重级别		缺陷的权值
高	→	4
中	→	3
低	→	2
轻微	→	1

加权法度量缺陷

- 例如，测试人员A在某个项目的测试中发现的缺陷数如下表所示。

缺陷严重级别	缺陷个数
高	100
中	200
低	300
轻微	400

- 例如，测试人员B在某个项目的测试中发现的缺陷数如下表所示。

缺陷严重级别	缺陷个数
高	150
中	350
低	300
轻微	200

加权法度量缺陷

- 如果将两位测试人员的测试结果通过加权的方式进行计算。测试人员A的计算结果如下表所示。

缺陷严重级别	缺陷个数	权值	缺陷价值
高	100	4	400
中	200	3	600
低	300	2	600
轻微	400	1	400
总计：2000			

加权法度量缺陷

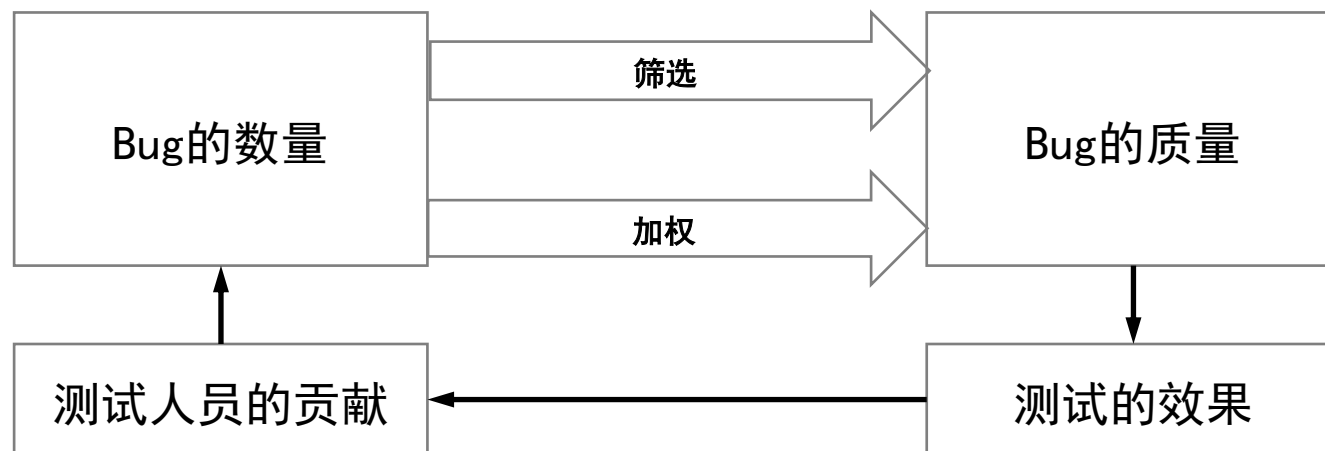
- 测试人员B的计算结果如下表所示。

缺陷严重级别	缺陷个数	权值	缺陷价值
高	150	4	600
中	350	3	1050
低	300	2	600
轻微	200	1	200
总计：2450			

- 所以，虽然二者发现缺陷的总数相等，但是通过加权计算可知缺陷的总体价值不一样。此外，还可以考虑其他的加权因素，例如缺陷的类型、缺陷发现的及时性、缺陷的重现率等。

加权法度量缺陷

- 加权法虽然科学，但是如果基于未加过滤的Bug来计算，则会多少有些不公平。例如，不同的测试人员对于Bug的严重程度的理解可能存在偏差。那么有没有什么解决的办法呢？
- **解决办法**：制定缺陷级别评估规范，用于指导测试人员进行Bug等级的划分。此外，每个缺陷的等级划分应该得到评审，对于不符合划分标准的应予以纠正。对于那些经确认拒绝处理的Bug，也不能纳入统计范畴。



目录

CONTENTS

2.1 度量Bug的数量

2.2 加权法度量缺陷

2.3 Bug的定性评估

2.4 Bug综合评价模型

2.5 测试覆盖率统计

Bug的定性评估

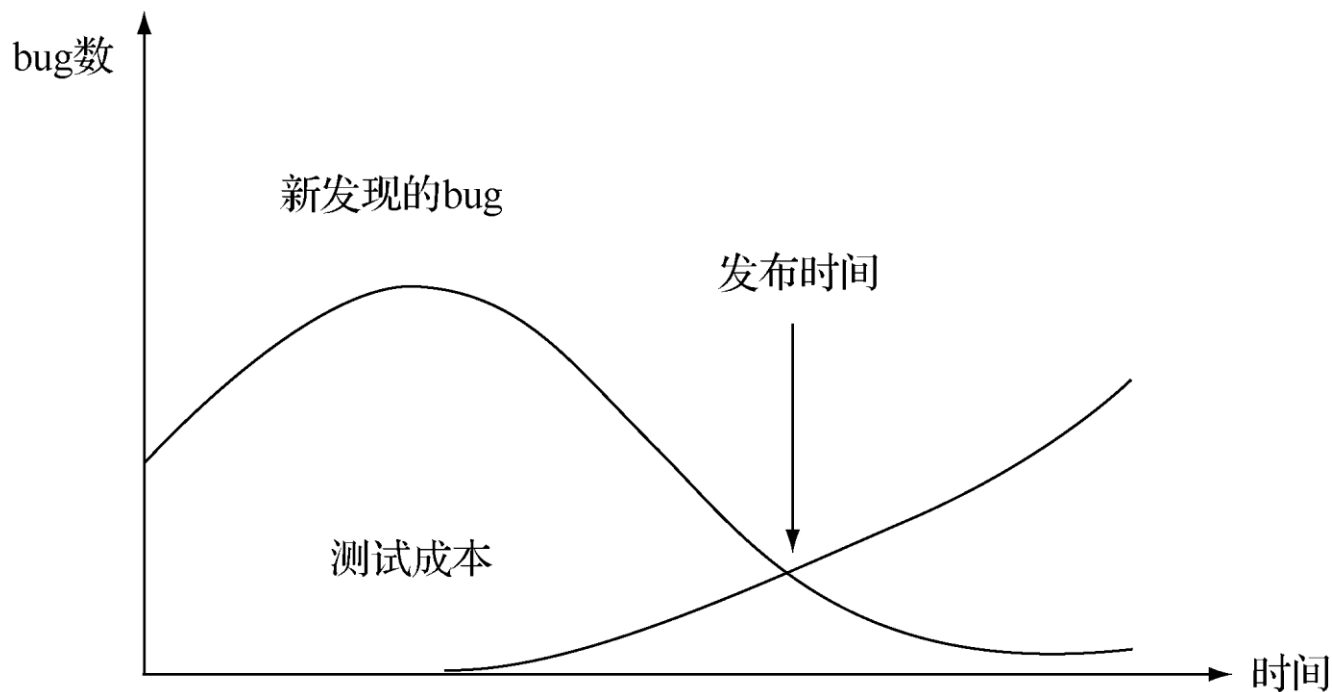
- 对于软件缺陷分析，常用的主要参数有以下4个：
 - 状态：软件缺陷的当前状态。
 - 优先级：必须处理和解决的软件缺陷的相对重要性。
 - 严重性：对最终用户、组织或者第三方的影响等。
 - 起源：导致软件缺陷的起源故障以及其位置，或者排除该软件缺陷需要修复的构件。

Bug的定性评估

- 软件测试的软件缺陷评估可以依据以下5类进行度量：

1. 软件缺陷发现率：

将发现的软件缺陷数量作为时间的函数来评估。创建软件缺陷趋势图和报告，如图所示。



Bug的定性评估

2. 软件缺陷潜伏期:

软件缺陷潜伏期是一种特殊类型的软件缺陷分析度量。软件缺陷潜伏期报告显示软件缺陷处于特定状态下的时间长短。实际测试工作中，发现越晚，危害就越大，修复成本就越高。

3. 软件缺陷分布:

软件缺陷分布是一种以平均值来估算软件缺陷的分布值。程序代码通常是以千行为单位，软件缺陷分布度量使用下面的公式计算：

$$\text{软件缺陷密度} = \frac{\text{软件缺陷数量}}{\text{代码行或功能点的数量}}$$

Bug的定性评估

4. 整体软件质量、软件缺陷注入率和清除率：

设F为描述软件规模的功能点；D1为软件开发过程中发现的所有软件缺陷数；D2为软件使用后发现的软件缺陷数；D为发现软件缺陷的总数，则 $D=D1+D2$ 。

对于一个软件项目，可以从不同角度来估算软件的质量、软件缺陷注入率和清除率：

$$\text{软件质量（每个功能点的软件缺陷数）} = \frac{D2}{F}$$

$$\text{软件缺陷注入率} = \frac{D}{F}$$

$$\text{整体软件缺陷清除率} = \frac{D1}{F}$$

Bug的定性评估

5. 整体软件缺陷清除率:

- ① 一级、二级bug修复率达到100%。
- ② 三级、四级bug修复率达到80%以上。
- ③ 五级bug修复率应该达到60%以上。

Bug的定性评估

- 除了必要的定量软件缺陷价值评估外，还可以加入定性的评估。
- 定性评估是指对测试人员发现的软件缺陷质量进行相对主观的衡量，可包括以下方面的评价：
 - 软件缺陷的类型分布。
 - 软件缺陷重现率。
 - 软件缺陷录入的清晰程度、简明程度等。
 - 软件缺陷的新颖性。

目 录

CONTENTS

2.1 度量Bug的数量

2.2 加权法度量缺陷

2.3 Bug的定性评估

2.4 Bug综合评价模型

2.5 测试覆盖率统计

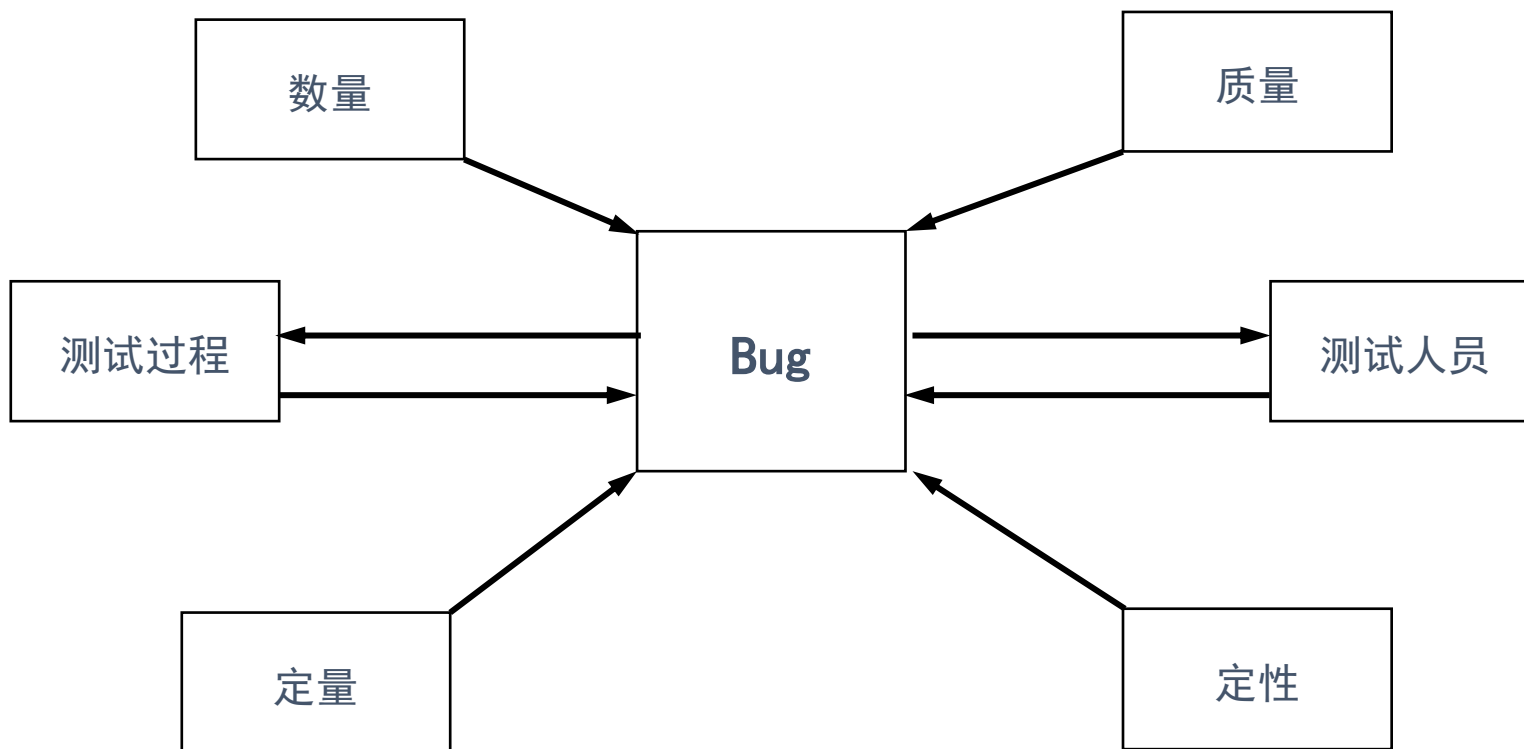
Bug综合评价模型

- 一个合格的软件缺陷报告应该包括完整的内容，至少包括：



Bug综合评价模型

- 在加入定性的评估后，可以形成一个如下图所示的综合评价模型



目录

CONTENTS

2.1 度量Bug的数量

2.2 加权法度量缺陷

2.3 Bug的定性评估

2.4 Bug综合评价模型

2.5 测试覆盖率统计

测试覆盖率统计

- 统计测试的覆盖率是一种衡量测试工作的方法。
- 只有测试充分覆盖了产品的各方面，才能发现尽可能多的Bug，才能给项目经理足够的信息去告诉用户放心地使用这个产品。
- 测试覆盖率可分为**代码行覆盖**、**功能模块覆盖**、**需求覆盖**等统计方式。

测试覆盖率统计

➤ 代码行覆盖率

- 代码行覆盖率是指测试执行遍历了代码的哪些区域，测试执行经过的代码行数与总的代码行数的比例。可以使用以下公式计算代码行覆盖率。

$$\text{代码行覆盖率} = (\text{已执行测试的代码行} / \text{总的代码行}) * 100\%$$

- 代码测试覆盖率对于一些在安全性上要求比较高的软件系统来说是十分重要的。代码行覆盖率可借助一些工具来实现统计，例如AQTime, DevPartner, Clover.Net等。

测试覆盖率统计

➤ 代码行覆盖率

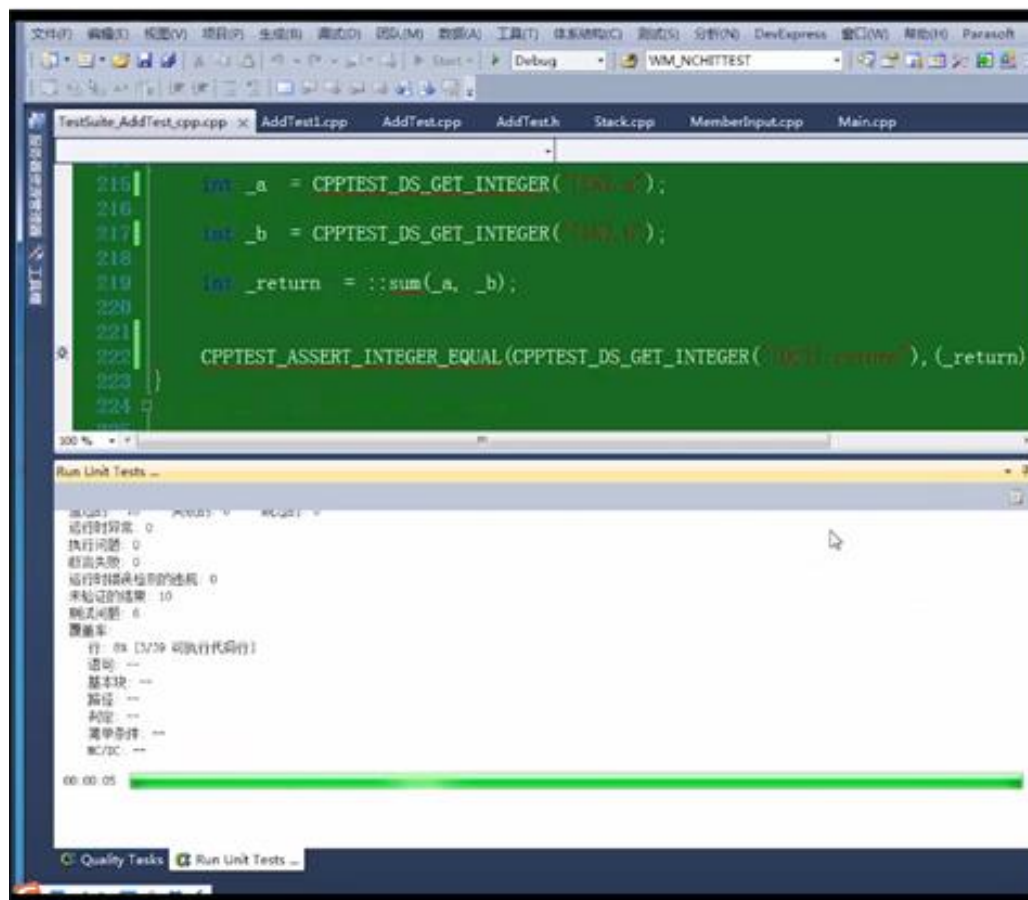
- 代码覆盖程度的度量方式是有很多种，这里介绍一下最常用的几种：

- ① **语句覆盖**：它度量程序中每条语句是否被测试到了。
- ② **判定覆盖**：又称分支覆盖、所有边界覆盖、基本路径覆盖。它度量程序中每个判定的分支是否都被测试到了。
- ③ **条件覆盖**：它度量判定中的每个子表达式结果true和false是否都被测试到了。
- ④ **路径覆盖**：又称断言覆盖。它度量函数的每个分支是否都被执行了。测试期望所有可能的分支都执行一遍，有多个分支嵌套时需要对多个分支进行排列组合，可想而知，测试路径随着分支数量的增加而呈指数级别增加。

测试覆盖率统计

➤ 代码行覆盖率

- 如图，用C++test得到代码行覆盖率的结果：



测试覆盖率统计

➤ 代码行覆盖率

- 代码行覆盖率只能代表测试过哪些代码，不能代表是否测试好这些代码。不能追求过高的代码行覆盖率，因为有些代码只有在非常罕见的情况下才能出现。
- 因而，测试人员不能盲目追求代码行覆盖率，而应该想办法**设计更多更好的测试用例**，哪怕多设计出来的测试用例对代码行覆盖率一点影响也没有。

测试覆盖率统计

➤ 代码行覆盖率

- 有些异常情况是很难出现的，如“OutOfMemoryException”；有些异常则不会出现，如果程序代码写得正确，如“DivideByZeroException”，那么这些异常相对应的代码就很可能不会被测试执行到。

测试覆盖率统计

➤ 代码行覆盖率

```
Try
{
    ... //
}
Catch (IOException IOex)
{
    ... // I/O 错误的异常
//
}
Catch (DataException DataEx)
{
    // 数据访问异常
    ... //
}
Catch (ArithmeticException ArEx)
{
    ... //
}
Catch (OutOfMemoryException MercyEx)
{
    //没有足够的内存继续执行程序时的异常
    ... //
}
```

测试覆盖率统计

➤ 代码行覆盖率

- 代码行覆盖率只能作为测试充分程度的参考，因为即使代码行覆盖率达到100%也很可能是测试不充分的。例如，下面的示例代码：

```
If(a==1 || b==1)
{
    MessageBox.show("OK!");
}
```

- 如果变量a和b是输入参数，那么只要a或者b有一个等于1就可以覆盖所有代码行。但是其他使用到a或b的地方则有可能受到不同取值的影响而产生不同的结果。如果仅仅满足于代码覆盖，那么测试显然是不够充分的。

测试覆盖率统计

➤ 功能模块覆盖率

- 功能模块覆盖率是一种比较粗的衡量方式。主要用在系统功能上，或者包括很多子系统、子模块的产品上，并且通常在回归测试时衡量测试的覆盖面。计算公式为：

$$\text{功能覆盖率} = (\text{已执行测试的功能模块数} / \text{总的功能模块数}) * 100\%$$

测试覆盖率统计

➤ 功能模块覆盖率

- 在制定功能模块覆盖率的衡量标准时，需要注意系统的各个功能模块之间是有关联的。
- 例如，测试人员在测试库存模块时，可能需要在基础配置模块中先初始化一些库存信息，而这就同时测试了基础配置模块的一部分功能；另外，有些模块在单元测试中已经详细地测试，且核心代码已经受控，则没有必要每次都进行详细的测试，因此不能每次都要求具有很高的功能模块覆盖率。

测试覆盖率统计

➤ 功能模块覆盖率

- 假设某个项目包括 m 个主要功能模块，在某次测试中，测试人员对其中的 n 个功能模块进行了测试，其他功能模块并未进行测试，则可统计出功能模块覆盖率为 n/m 。
- 这种情况下统计功能模块覆盖率是没有意义的。

测试覆盖率统计

➤ 数据库覆盖率

- 介于代码行覆盖和功能模块覆盖率之间。数据库覆盖率指的是测试人员测试的功能模块对数据库表的访问面积的覆盖率。
- 这种覆盖率的计算方法只能应用在数据库软件系统的测试覆盖率统计上。统计的方法是在测试过程中跟踪程序访问数据库的操作产生的SQL语句，然后根据SQL语句覆盖到的表存储过程、视图、函数、触发器等数据库对象的面积来统计测试覆盖率。

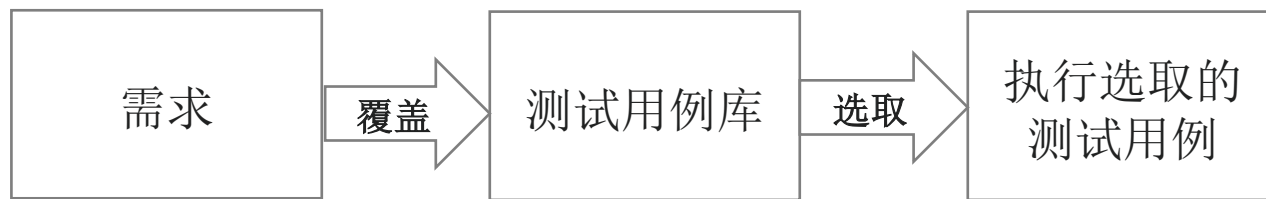
$$\text{数据库覆盖率} = (\text{SQL中出现的数据库的对象数} / \text{数据库总的对象数}) * 100\%$$

测试覆盖率统计

➤ 需求覆盖率

- 需求覆盖率是基于需求项的覆盖度量，主要通过分析测试用例的执行情况来衡量对需求的满足程度。计算公式为：

$$\text{需求覆盖率} = (\text{被验证到的需求数量} / \text{总的需求数量}) * 100\%$$



- 需求覆盖率能够较好地体现**测试的覆盖率和测试人员的工作效率**，但是这种统计方式要求比较规范的测试过程，需求必须是相对完整覆盖用户要求的。测试人员基于需求设计出测试用例，纳入测试用例库，并且需要不断地维护测试用例库，使其能体现测试的需求。

目 录

CONTENTS

01

软件测试度量简介

02

度量方法及其应用

03

常见的度量类型

04

小结

常见的测试度量类型

- 不同的软件测试度量如下图所示

手工测试度量	性能测试度量	自动化测试度量	通用度量
● 测试用例生产率 ● 测试执行摘要 ● 软件缺陷可接受率 ● 软件缺陷不接受率 ● 不良软件缺陷修复率 ● 测试执行生产率 ● 测试效率 ● 软件缺陷严重程度指数	● 性能脚本生产率 ● 个性执行摘要 ● 性能执行数据—客户端 ● 性能执行数据—服务器端 ● 性能测试效率 ● 性能严重程度指数	● 自动化脚本生产率 ● 自动化测试执行生产率 ● 自动化覆盖率 ● 成本对比	● 工作量偏差 ● 进度偏差 ● 范围变化

目 录

CONTENTS

3.1 手工测试度量

3.2 性能测试度量

3.3 自动化测试度量

3.4 通用度量

手工测试度量

(1) 测试用例生产率

- 该度量基于测试用例编写的生产率，这些测试用例有确定的结果。
测试用例生产率（Test Case Productivity, TCP）的计算公式如下：

$$\text{测试用例生产率} = \frac{\text{总原始测试步骤}}{\text{工作时间 (小时)}} \quad (\text{单位: 步骤/小时})$$

手工测试度量

测试例子：

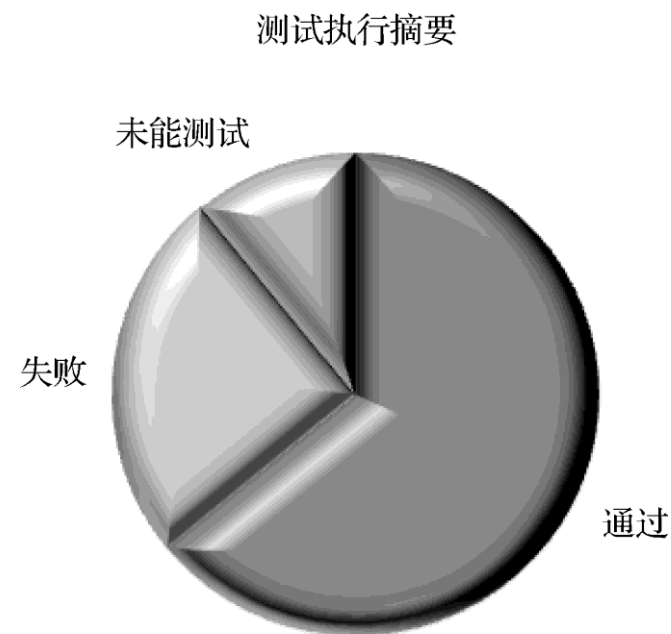
测试用例名称	测试步骤
XYZ_1	30
XYZ_2	32
XYZ_3	40
XYZ_4	36
XYZ_5	45
总步骤	183

- 结论为8小时编写183个测试步骤，则 $TCP=183/8\approx22.8$ ，因此可以知道测试用例生产率为23步骤/小时。

手工测试度量

(2) 测试执行摘要

- 测试执行摘要 (Test Execution Summary) 给出测试用例执行结果分类方面的状态及原因, 针对各类测试用例, 给出了发布版本的静态视图, 并收集执行结果及测试用例数量的数据。
- 测试执行摘要如图所示:
- **摘要趋势**: 人们也可以为各种不能进行的测试以及失败的测试用例的原因进行分类以展示同样的趋势。



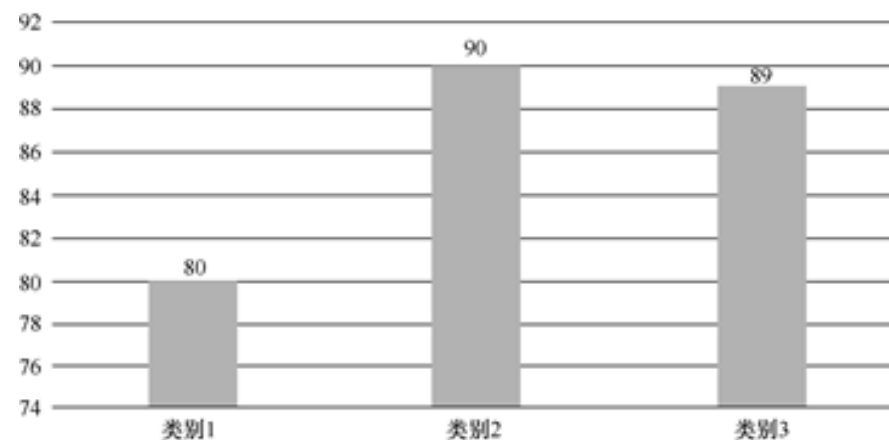
手工测试度量

(3) 软件缺陷可接受率

- 软件缺陷可接受率（Defect Acceptance, DA）决定测试组在执行期间定义的有效软件缺陷的数量，其计算公式如下：

$$\text{软件缺陷可接受率} = \frac{\text{有效软件缺陷数}}{\text{总软件缺陷数}} \times 100\%$$

- 度量值可以和以前发布版本对比分析，如图所示。



软件缺陷可接受趋势

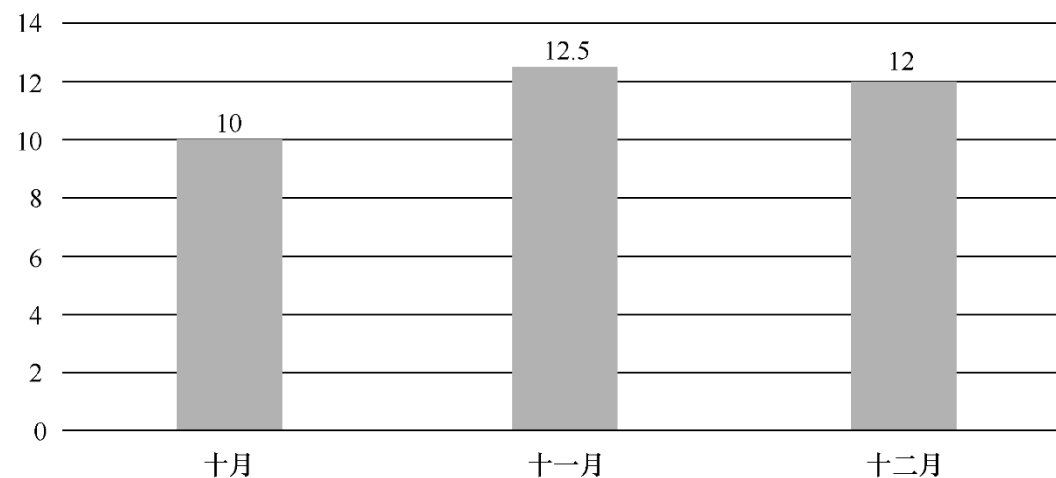
手工测试度量

(4) 软件缺陷不接受率

- 软件缺陷不接受率（Defect Rejection, DR）决定在测试期间不接受的软件缺陷数量，其计算公式如下：

$$\text{软件缺陷不接受率} = \frac{\text{软件缺陷不接受数}}{\text{总软件缺陷数}} \times 100\%$$

- 它提供了测试组已经打开的无效软件缺陷的百分比，如右图所示。



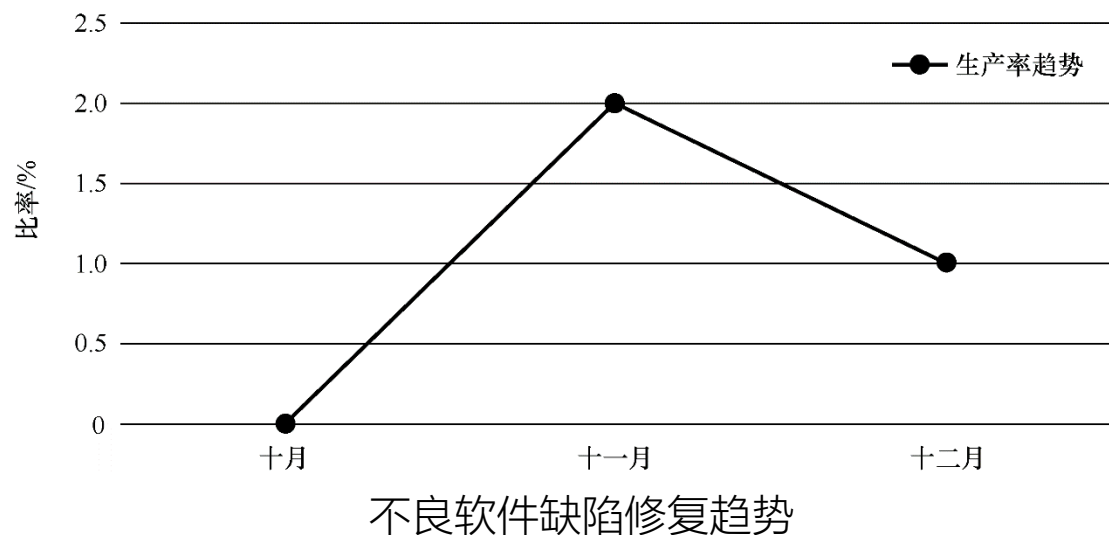
软件缺陷不接受趋势

(5) 不良软件缺陷修复率

- 不良软件缺陷修复率 (Bad Fix Defect) 是指由解决缺陷导致的新软件缺陷。这项度量决定软件缺陷修复过程的效果，其计算公式如下：

$$\text{不良软件缺陷修复率} = \frac{\text{不良软件缺陷修复数}}{\text{总有效软件缺陷数}} \times 100\%$$

- 它指出了需要控制的不良软件缺陷修复的百分比，如图所示。



(6) 测试执行生产率

- 进一步分析测试执行生产率 (Test Execution Productivity, TEP) 可以得出确切的结果, TEP的计算公式如下:

$$\text{测试执行生产率} = \frac{\text{测试用例执行数}}{\text{执行时间 (小时)}} \times 8 \text{ (单位: 执行次数/天)}$$

- 测试执行数 (Number of Test Case Executed, TE) 的计算方法如下:

$$\text{TE} = \text{BTC} + \text{T}(0.33) \times 0.33 + \text{T}(0.66) \times 0.66 + \text{T}(1) \times 1,$$

- 其中, BTC是指基本测试用例 (Base Test Case), 即至少执行了一次的测试用例的数量。

手工测试度量

(6) 测试执行生产率

- 进一步分析测试执行生产率 (Test Execution Productivity, TEP) 可以得出确切的结果, TEP的计算公式如下:

$$\text{测试执行生产率} = \frac{\text{测试用例执行数}}{\text{执行时间 (小时)}} \times 8 \text{ (单位: 执行次数/天)}$$

- T (1) =重新测试需执行总TC步骤的71%至100%的TC数量
- T (0.66) =重新测试需执行总TC步骤的41%至70%的TC数量
- T (0.33) =重新测试需执行总TC步骤的1%至40%的TC数量

手工测试度量

➤ 基本测试用例情况

用户名称	基础执行效果 (hr)	重复运行情况1	重复执行效率1 (hr)	重复运行情况2	重复执行效率2 (hr)	重复运行情况3	重复执行效率3 (hr)
XYZ_1	2	T (0.66)	1	T (0.66)	0.45	T (1)	2
XYZ_2	1.3	T (0.33)	0.03	T (1)	2		
XYZ_3	2.3	T (1)	1.2				
XYZ_4	2	T (1)	2				
XYZ_5	2.15						

手工测试度量

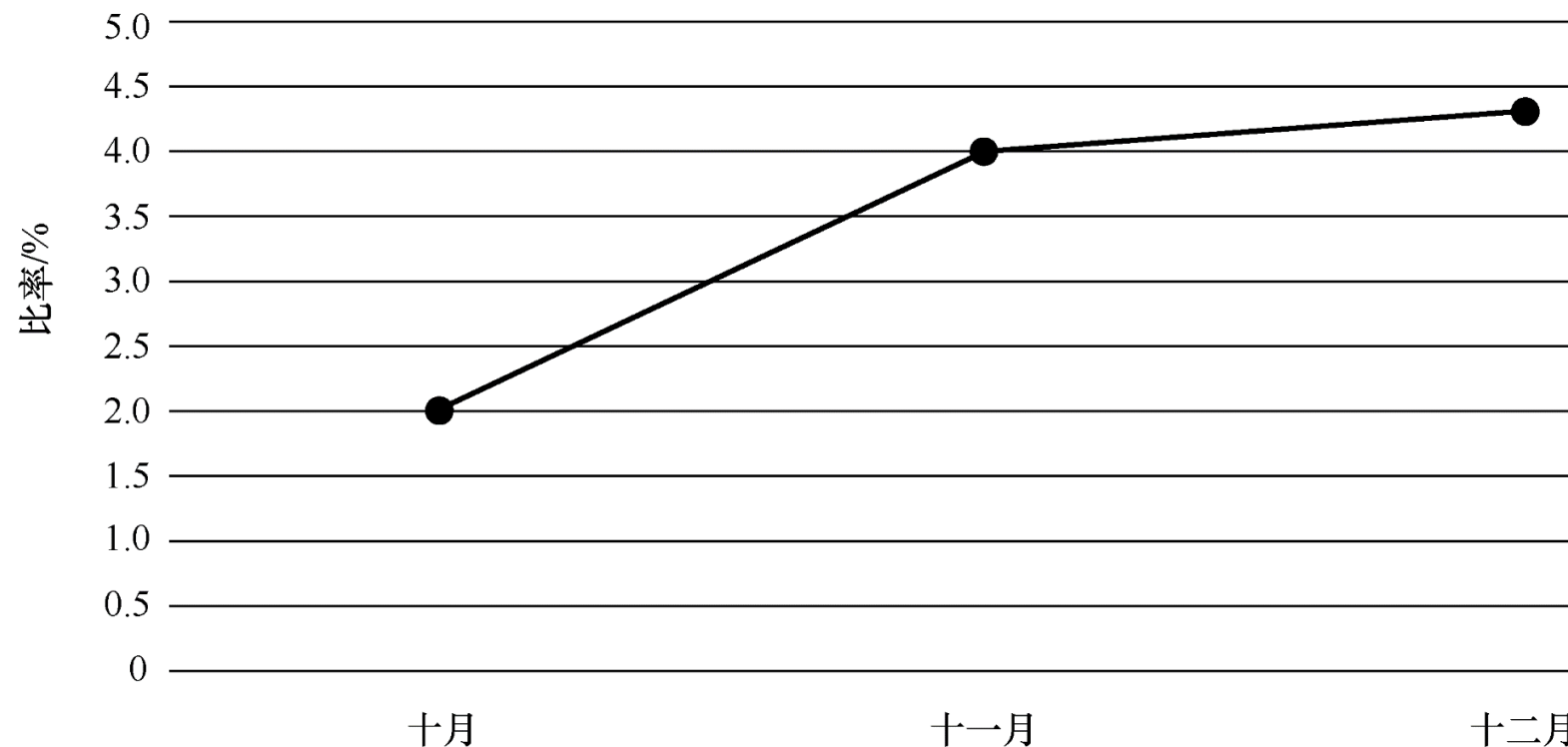
➤ 基本测试用例统计

基础测试用例	5
T (1)	4
T (0.66)	2
T (0.33)	1
测试用例执行时间TE	19.7

- 因此，可以得出 $T_e = 5 + (1 \times 4 + 2 \times 0.66 + 1 \times 0.33) = 5 + 5.65 = 10.65$ ，测试执行生产率 = $10.65 / 19.7 \times 8 \approx 4.3$ （执行次数/天）。

手工测试度量

➤ 基本测试用例统计



- 人们可以和以前发布版对比测试执行生产率，从而得出有效结论，如上图所示。

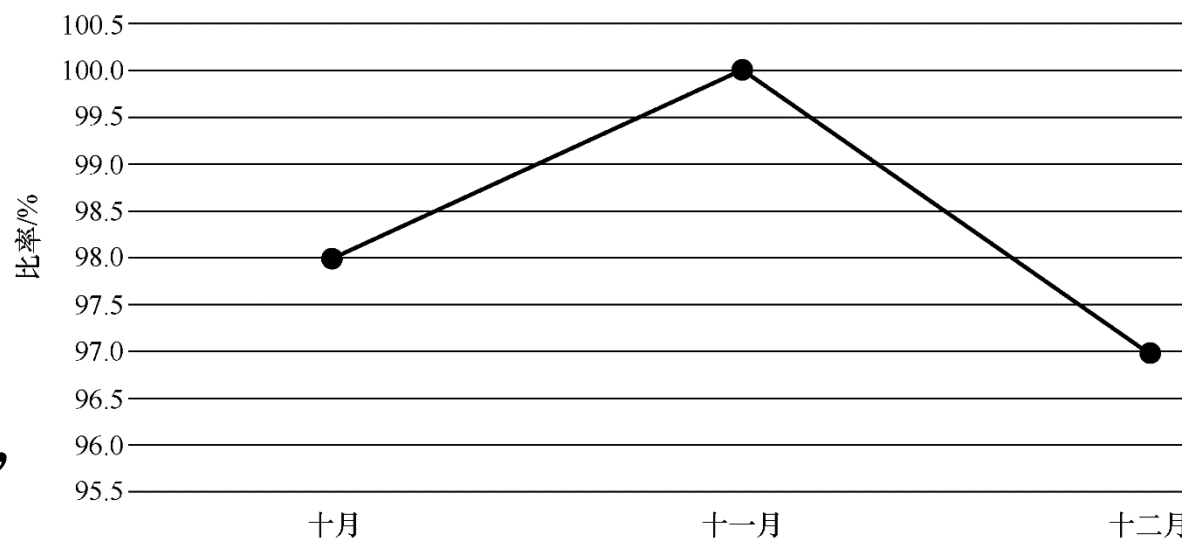
手工测试度量

(7) 测试效率

- 测试效率 (Test Efficiency, TE) 决定测试组在提交软件缺陷时的效率，其计算公式如下：

$$\text{测试效率} = \text{DT} / (\text{DT} + \text{DU}) \times 100\%$$

- DT为在测试期间定义的有效软件缺陷数；
- DU为应用发布后由用户定义的有效软件缺陷数。换句话说就是，事后测试软件缺陷，如图所示。



测试效率趋势

(8) 软件缺陷严重度指数

- 软件缺陷严重度指数 (Defect Severity Index, DSI) 决定测试时和发布时的产品质量。基于这项度量，人们可以决定是否发布产品，即这项度量代表了产品质量，其计算公式如下：

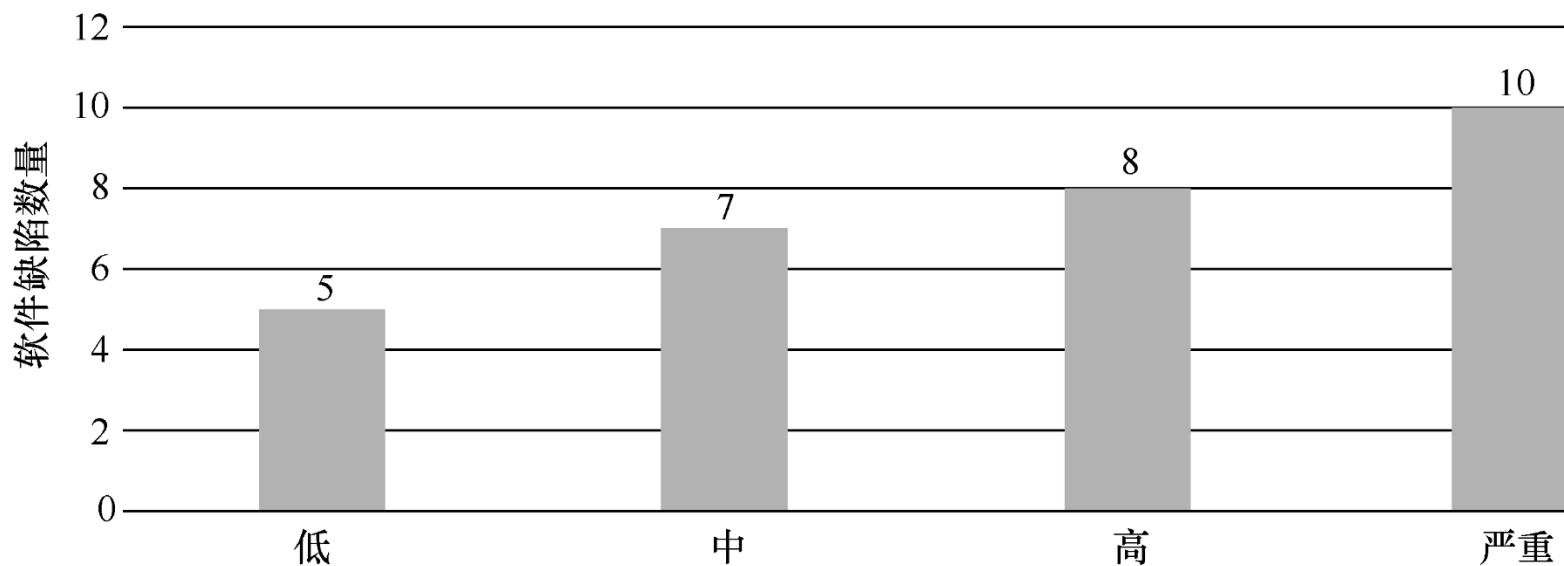
$$\text{软件缺陷严重度指数} = \frac{\sum(\text{缺陷严重度指数} \times \text{该缺陷严重度指数下的有效软件缺陷数量})}{\text{有效软件缺陷总数}}$$

- 可以将软件缺陷严重程度分为以下两个部分：
 - ① 所有状态软件缺陷的严重度指数：这项值提供了在测试中的产品质量。
 - ② 打开状态软件缺陷的严重度指数：这项值给出发布时的产品质量。此时计算软件缺陷严重程度，必须考虑仅仅是打开状态的软件缺陷。

(8) 软件缺陷严重程度指数

$$\text{DSI (打开状态)} = \frac{\sum(\text{缺陷严重程度指数} \times \text{该缺陷严重程度指数下打开状态的有效软件缺陷数量})}{\text{有效打开状态的软件缺陷总数}}$$

- 如下图所示是对于所有状态的缺陷严重程度指数为2.8的DSI

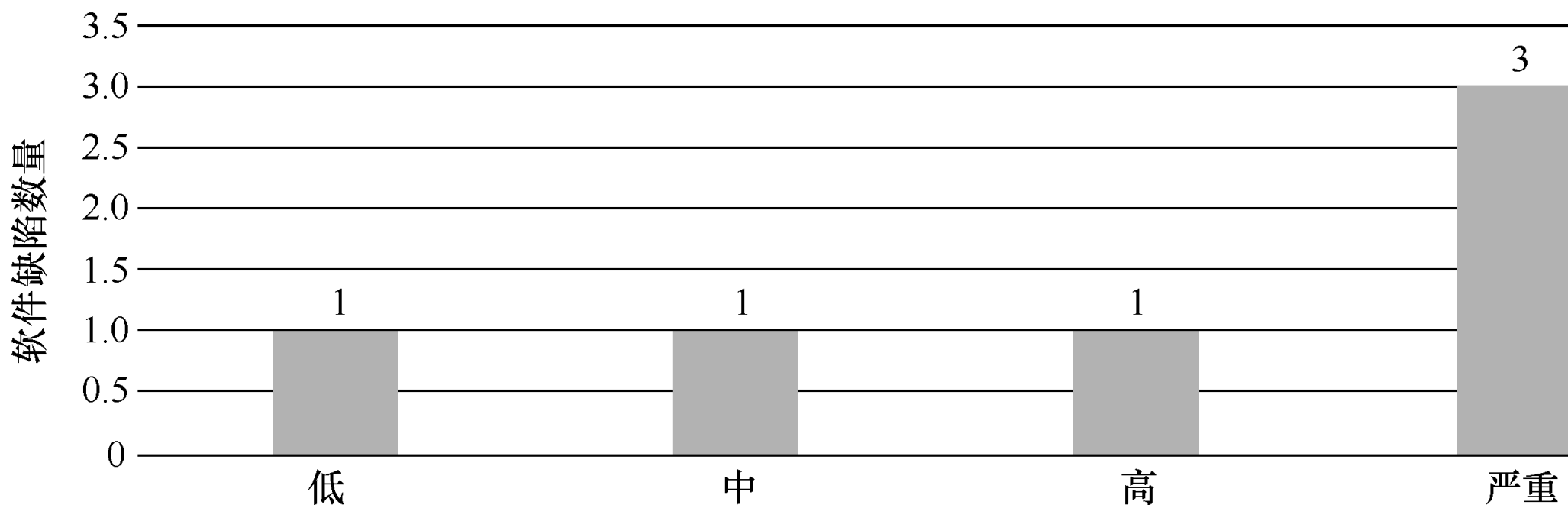


(8) 软件缺陷严重度指数

①测试中的产品质量，即所有状态软件缺陷的严重度指数=2.8(高严重程度)

②发布时的产品质量，即打开状态软件缺陷的严重度指数=3.0(高严重程度)

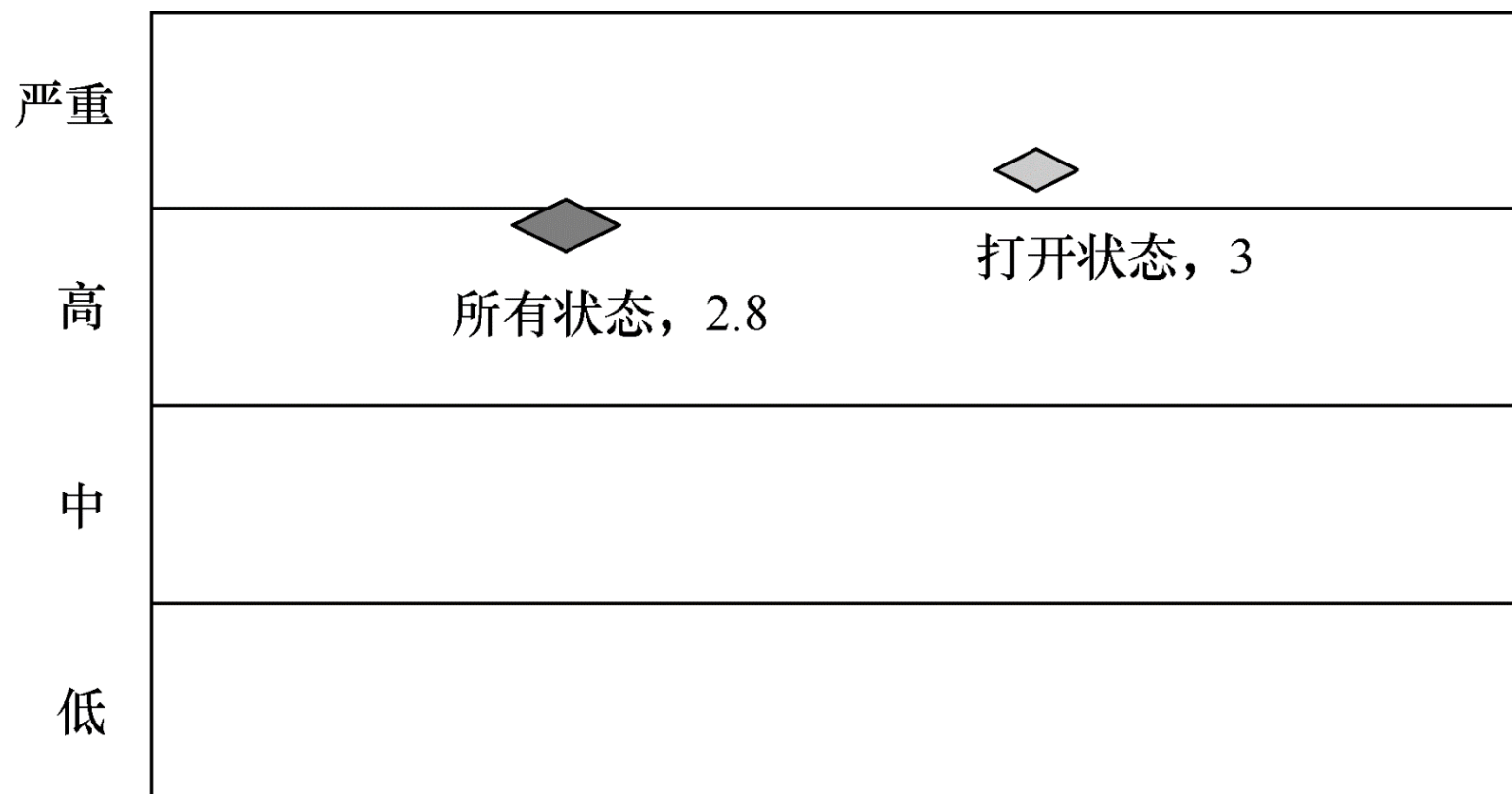
● 如下图所示是对于打开状态的缺陷严重度指数为3.0的DSI



手工测试度量

(8) 软件缺陷严重度指数

- 软件缺陷严重度指数的分布如下图所示：



目 录

CONTENTS

3.1 手工测试度量

3.2 性能测试度量

3.3 自动化测试度量

3.4 通用度量

性能测试度量

(1) 性能脚本生产率

- 性能脚本生产率（Performance Scripting Productivity, PSP）为性能测试脚本提供脚本生产率以及一段时间内的趋势，其计算公式如下：

$$\text{性能脚本生产率} = \frac{\Sigma \text{性能操作}}{\text{用时 (小时)}}$$

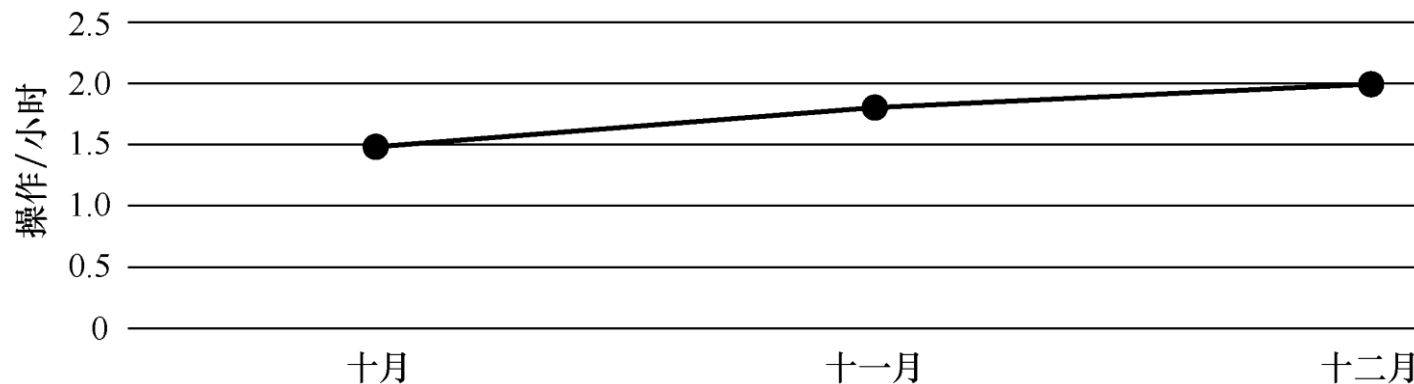
性能测试度量

(1) 性能脚本生产率

- 性能脚本示例执行的操作是：①点击数量，即点击刷新数据；②输入参数的数量；③关联参数数量。

执行性能	计数
点击数量	10
输入参数数量	5
关联参数数量	5
总执行性能	20

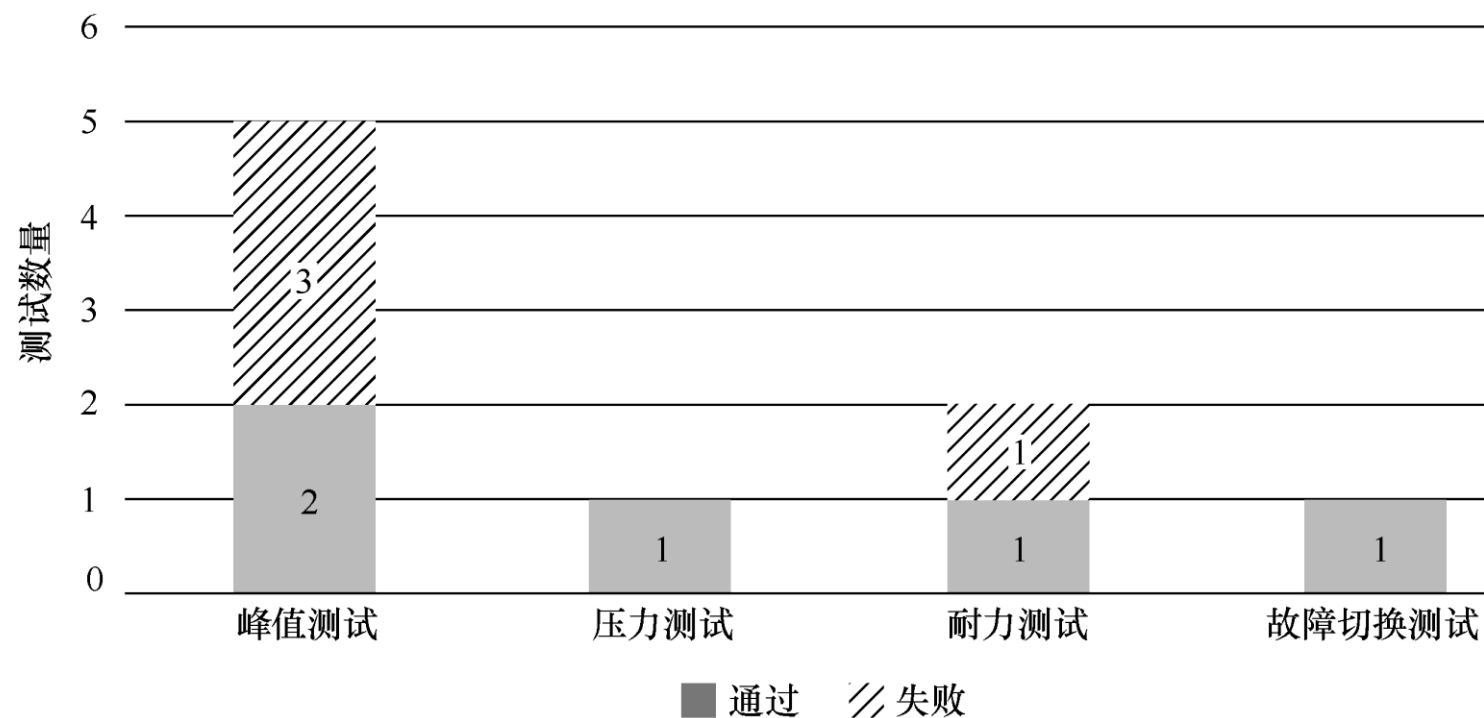
- 脚本编写用时=10小时，性能脚本生产率= $20/10=2$ （操作/小时），如下图：



性能测试度量

(2) 性能执行摘要

- 性能执行摘要（Performance Execution Summary）列出了针对性能测试的各种类型，由状态（通过/失败）控制的测试数量。性能测试类型包括峰值测试、压力测试、耐力测试、故障切换测试等，如图所示。



性能测试度量

(3) 性能执行数据—客户端

- 性能执行数据—客户端给出执行性能测试时客户端数据的详细信息。这项度量的数据包括运行用户数、响应时间、每秒点击率、吞吐量、每秒总事务数、第1个字节传输时间、每秒错误数。

(4) 性能执行数据—服务器端

- 性能执行数据—服务器端给出执行性能测试时服务器端数据的详细信息。这项度量的数据包括CPU占用率、内存占用率、堆内存占用率、每秒数据库连接数。

性能测试度量

(5) 性能测试效率

- 性能测试效率（Performance Test Efficiency, PTE）决定了性能测试团队满足需求的质量，如果需要，可以将其用作后续改进的输入，其计算公式如下：

$$\text{性能测试效率} = \frac{\text{PT期间满足的需求}}{\text{PT期间满足的需求} + \text{PT退出后未满足的需求}} \times 100\%$$

- PT为性能测试期间。
- 该指标需要在性能测试期间及测试结束后收集数据点。

(5) 性能测试效率

- 一些性能测试的需求如下：平均响应时间、每秒事务数、可以处理预定义的最大用户负载、服务器稳定性。
- 例如，考虑在性能测试期间需满足上述需求。
- 已知：性能测试期间的需求数= 4；在产品中，平均响应时间比期望值更好，在性能测试结束后没有满足需求=1；
- 可知， $PTE = 4 / (4+1) \times 100\% = 80\%$ ，即性能测试效率是80%。

(6) 性能严重程度指数

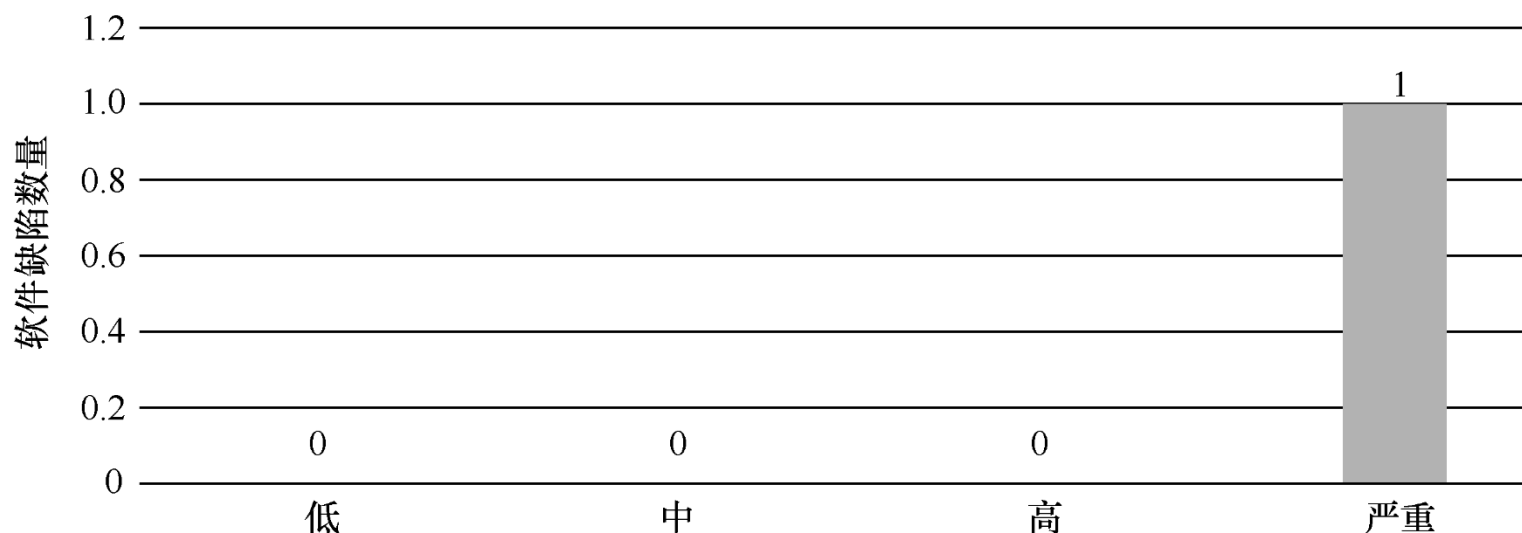
- 性能严重程度指数 (Performance Severity Index, PSI) 决定基于性能标准的产品质量，性能标准可以决定下阶段是否发布产品，即它代表性能方面测试的产品质量。其计算公式如下：

$$\text{性能严重程度指数} = \frac{\sum(\text{严重指数} \times \text{该严重级别未满足的需求数})}{\text{未满足需求总数}}$$

性能测试度量

(6) 性能严重程度指数

- 如果性能不满足要求，则可以分配要求的严重性，以便根据性能来决定产品的发布。
- 例如：考虑到平均响应时间没有满足的重要需求，测试人员可以按照标准打开软件缺陷严重程度。性能严重程度指数 $= (4 \times 1) / 1 = 4$ （严重），如图所示。



目 录

CONTENTS

3.1 手工测试度量

3.2 性能测试度量

3.3 自动化测试度量

3.4 通用度量

自动化测试度量

(1) 自动化脚本生产率

- 自动化脚本生产率（Automation Scripting Productivity, ASP）为基于已有的分析得出最有效结论的自动化测试脚本生产率，其计算公式如下：

$$\text{自动化脚本生产率} = \frac{\Sigma \text{执行操作}}{\text{用时 (小时)}} \quad (\text{单位: 操作/小时})$$

- 自动化脚本示例执行的操作如下：

- ① 点击数量，即点击刷新的数据。
- ② 输入参数的数量。
- ③ 增加的检查点个数。

自动化脚本的操作和相关统计

操作	计数
点击数量	10
输入参数数量	5
增加的检查点个数	10
总的操作性能	25

脚本编写用时=10小时，ASP=25/10=2.5，
即自动化测试脚本生产率= 2.5（操作/小时）。

自动化测试度量

(2) 自动化测试执行生产率

- 自动化测试执行生产率（Auto Execution Productivity, AEP）给出自动化测试用例执行生产率，其计算公式如下：

$$\text{自动化测试执行生产率} = \frac{\text{自动化测试用例执行总数 (ATE)}}{\text{执行工作量 (小时)}} \times 8$$

- 其中，ATE的计算方法如下：

$$\text{ATE} = \text{BTC} + T(0.33) \times 0.33 + T(0.66) \times 0.66 + T(1) \times 1$$

自动化测试度量

(3) 自动化覆盖率

- 自动化覆盖率指出自动化手工测试用例的百分比，其计算公式如下：

$$\text{自动化覆盖度} = \frac{\text{自动化测试用例总数}}{\text{手动测试用例总数}} \times 100\%$$

- 例如，如果有100 个手动测试用例，其中60个用例可以自动化测试，那么自动化覆盖率为60%。

自动化测试度量

(4) 成本对比

- 成本对比给出在手动测试和自动化测试之间的成本比较。这项度量被用来得出确定的投资回报，手动测试成本评估如下：

$$\text{成本（手动测试）} = \text{执行结果（小时）} \times \text{支付比率}$$

- 自动化测试成本评估如下：

$$\begin{aligned} \text{成本（自动化测试）} = & \text{购买工具成本（一次性投资）} + \text{维护成本} + \text{脚本开发成本} \\ & + \text{执行结果} \times \text{支付率} \end{aligned}$$

- 如果脚本是重用的，脚本开发成本改成脚本更新成本。

目录

CONTENTS

3.1 手工测试度量

3.2 性能测试度量

3.3 自动化测试度量

3.4 通用度量

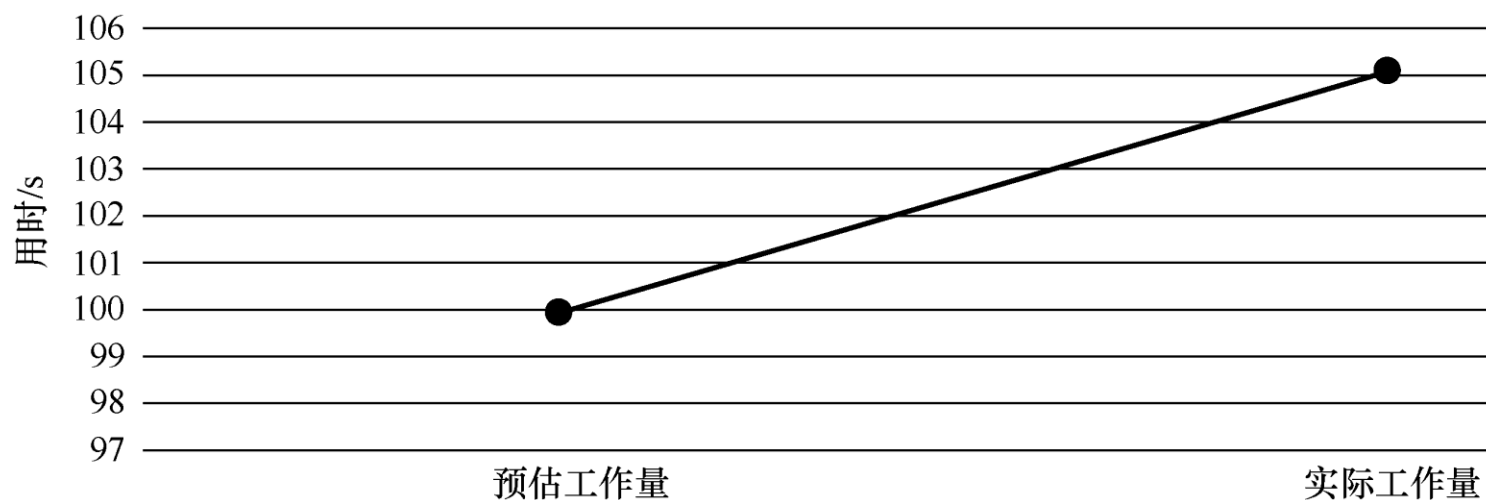
通用度量

(1) 工作量偏差

- 工作量偏差（Effort Variance, EV）用来指出估计结果的差异，其计算公式如下：

$$\text{工作量偏差} = \frac{\text{实际工作量} - \text{预估工作量}}{\text{预估工作量}} \times 100\%$$

- 工作量偏差趋势如图：



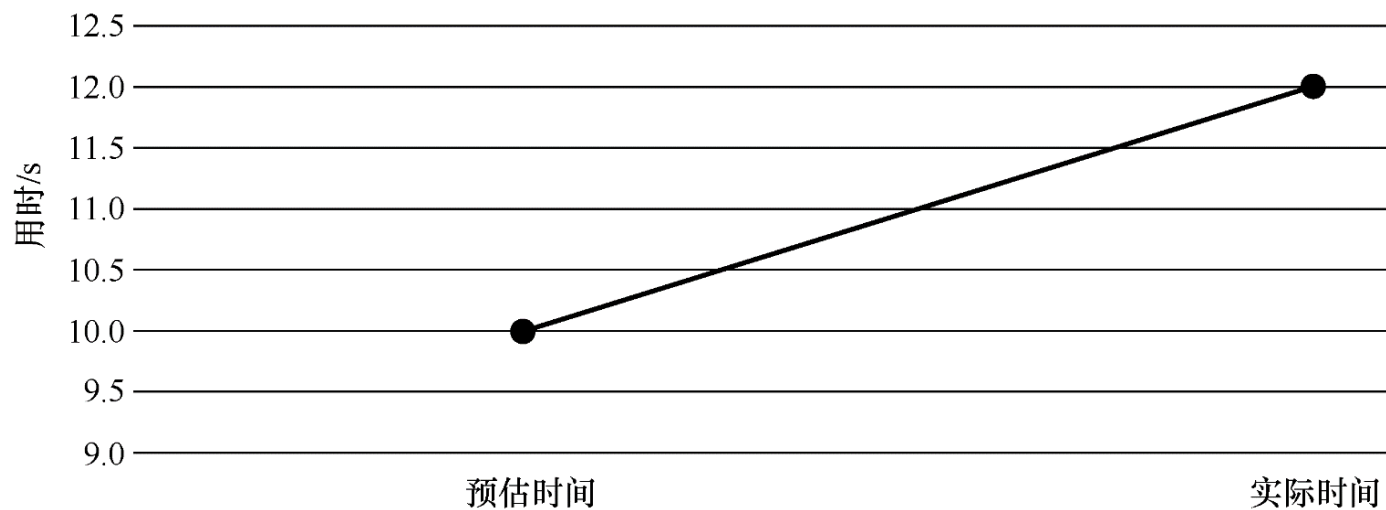
通用度量

(2) 进度偏差

- 进度偏差（Schedule Variance, SV）用来指出估计进度的差异，即日期数，其计算公式如下：

$$\text{进度偏差} = \frac{\text{实际时间} - \text{预估时间}}{\text{预估时间}} \times 100\%$$

- 进度偏差趋势如图：



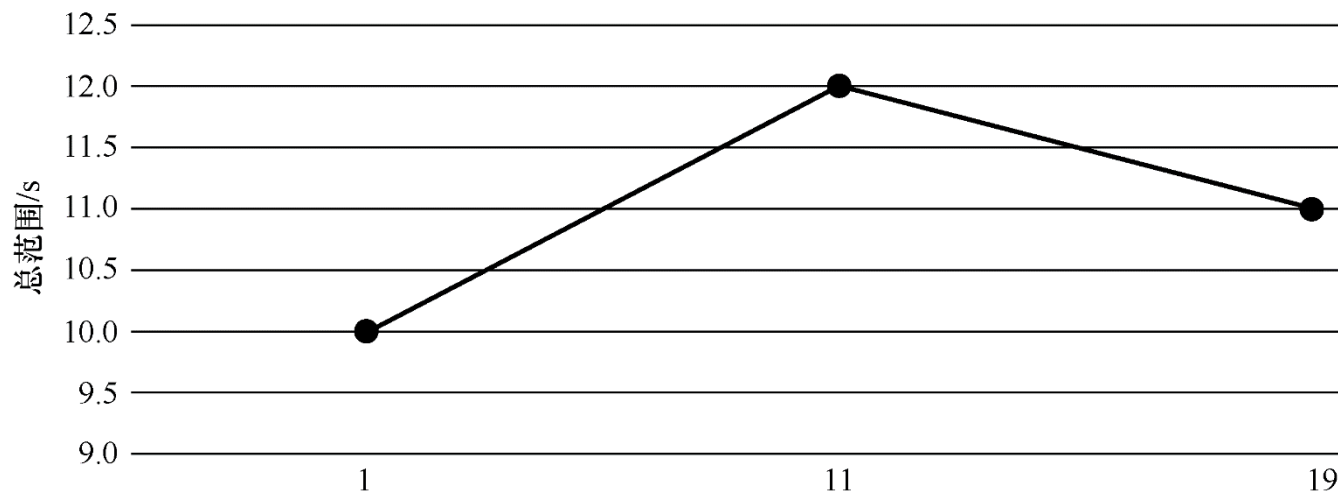
(3) 范围变化

- 范围变化 (Scope Change, SC) 用来指出如何固定测试范围，其计算公式如下：

$$\text{范围变化} = \frac{\text{总范围} - \text{以前范围}}{\text{以前范围}} \times 100\%$$

- 当范围扩大，总范围=以前的范围+新范围。
- 当范围缩小，总范围=以前的范围-新范围。

发布版本的范围变化趋势如图



目 录

CONTENTS

01

软件测试度量简介

02

度量方法及其应用

03

常见的度量类型

04

小结

小结

这个知识点主要学习内容：

- 软件测试度量的概念
- 度量方法及其应用
- 常见的度量类型

习题

1. 什么是软件测试的度量?
2. 软件测试的度量应该遵循哪些重要原则?
3. 软件缺陷综合评价模型包括哪6个方面?
4. 软件测试覆盖率可分为哪几个方面?
5. 简述软件测试度量的复杂性和经济性。